

Fall 2025: CS 141 Exam #3 (final)

Please print your name here: _____

This is an **open-note** exam. You are permitted to use written or printed notes, but not a computer. The last page is a reference for the APIs that we've used in class; we suggest that you tear it off to make it easier to use during the exam. Students whose native language is not English may use a dictionary. *Nothing else is permitted.*

If a problem is unclear to you, include a brief explanation of how you interpret it.

Show your work!

You will not get partial credit if we cannot figure out how you got your answer.

Do not talk to other students or look at other students' exams.

Please sign below to indicate that you have upheld the highest standards of Knox College's Honor Code, and that you have not cheated in any way.

1. **Types (10 points).** Suppose you have the following variable declarations.

```
boolean pikachu;  
char bulbasaur;  
String charmander;  
int squirtle;  
int[] eevee;
```

What are the **types** of the expressions below?

1. "Trainer: " + charmander
2. charmander.charAt(2)
3. !pikachu
4. bulbasaur + "!"
5. charmander + eevee[0]
6. new String[squirtle]
7. eevee.length
8. "Caught: " + pikachu
9. charmander.substring(1, 4)
10. charmander.substring(0, 1) + bulbasaur

2. **Kinds of variables (10 pts).** Suppose we have the following class declaration with a main method. Identify what kind of variable each of the **bold/underlined** variables are by writing the appropriate number next to each commented letter in the margin at the right. Every variable will be one of the 5 kinds listed below (instance, local, static, formal, actual). The first one (// A) is done for you.

1. instance variable
2. local variable
3. static variable
4. formal parameter
5. actual parameter

CODE	LABEL	KIND OF VAR
public class Wizard {		
<u>String spell;</u>	// A	1(instance)
<u>int powerLevel;</u>	// B	
static <u>int totalWizards</u> = 0;	// C	
public Wizard(<u>String spell</u> , int powerLevel) {	// D	
this.spell = spell;		
this.powerLevel = powerLevel;		
totalWizards++;		
}		
public void boostPower(<u>int amount</u>) {	// E	
powerLevel += amount;		
}		
public boolean isStrong() {		
<u>int remainder</u> = powerLevel % 5;	// F	
return remainder == 0;		
}		
public String castSpell() {		
return "Casting " + <u>spell</u> + "!";	// G	
}		
public static void main(String[] args) {		
<u>String s</u> = "Fireball";	// H	
<u>Wizard w1;</u>	// I	
w1 = new Wizard(<u>s</u> , 75);	// J	
boolean <u>strong</u> = w1.isStrong();	// K	
}		
}		

3. **Write a class (10 points).** Write a class in Java that represents a `Hero` in a video game.

A `Hero` has a `name`, `level`, `health points`, and `max health points`.

The class should include:

- a. A **constructor** that takes the `name`, `level`, and `health points` as parameters and sets them to instance variables.
- b. A **constructor** that takes the `name` and sets the `name` to the instance variable, sets the `level` to 1, and sets the `health points` to 10.
- c. A `toString()` method that returns the `Hero` information in the format: `"Name (Level X) - HP: Y"`.
- d. A **method** called `isStrongerThan()` that takes another `Hero` as a parameter and returns `true` if this `Hero`'s level is higher than the other's, and `false` otherwise.
- e. A **method** called `heal()` that takes `health points` as a parameter and adds to the `Hero`'s current health, but can't go over `max health`.
- f. A **method** called `damage()` that takes `health points` as a parameter and subtracts from the `Hero`'s current health, but can't go below 0.

4. Mystery function #1 (10 points). We have the function below.

```
public static String mystery1(String str) {  
    String result = "";  
    for (int i = 0; i < str.length(); i++) {  
        char ch = str.charAt(i);  
        int found = 0;  
        for (int j = 0; j < result.length(); j++) {  
            if (ch == result.charAt(j)) {  
                found++;  
            }  
        }  
        if (found == 0) {  
            result += ch;  
        }  
    }  
    return result;  
}
```

a. What does the function return given the following inputs?

i. `mystery1("abc")`

ii. `mystery1("aabac")`

iii. `mystery1("bbiirrd")`

b. Summarize what this method does in one sentence. ("Summarize" means to describe the overall effect; do not describe each line of the function)

5. Mystery functions #2 (10 points). Consider the function below:

```
public static int[] mystery2(int[] nums) {  
    int[] arr = new int[nums.length];  
    int total = 0;  
    for (int i = 0; i < nums.length; i++) {  
        total += nums[i]*nums[i];  
        arr[i] = total;  
    }  
    return arr;  
}
```

a. What does the function return given the following inputs?

i. `mystery2(new int[] {1})`

ii. `mystery2(new int[] {1,2,3,4})`

iii. `mystery2(new int[] {4,3,2,1})`

b. Summarize what this function does in one sentence. ("Summarize" means to describe the overall effect; do not describe each line of the function)

6. **Arrays (10 points).** Write a function (static method) called `countPowerDips` that takes an array of `int` called `nums` as a parameter and counts the number of “power dips” in the array. A power dip is a number that is less than the square of both of its neighbors.

For example, In the array **[3, 4, 2, 5, 1, 6]**, there are **2** power dips:

- a. 2 is less than 4^2 and less than 5^2
- b. 1 is less than 5^2 and less than 6^2

```
countPowerDips([3, 4, 2, 5, 1, 6]) => 2
countPowerDips([2, 1, 2])          => 1
countPowerDips([5, 30, 5])         => 0
countPowerDips([1, 3, 1, 3, 1])    => 1
```

```
public static int countPowerDips(int[] nums) {
```

```
}
```


7. **Strings (10 points).** Write a function (static method) called `getSurrounding` that takes a `String str` and an `int index` as parameters, and returns a new `String` containing the two characters that come immediately before and two characters after the character at the given index (**if they exist otherwise return empty: ""**). For example:

```
getSurrounding("planet", 2);    => "plne"  (before: 'pl', after:
'ne')
getSurrounding("mars", 0);      => "ar"      (only after 'ar')
getSurrounding("saturn", 5);    => "ur"      (only before 'ur')
getSurrounding("moon", 4);      => ""        (index invalid , so "")
```

```
public static String getSurrounding(String str, int index) {
```

8. **Recursion (10 points).** Write a recursive function called `charMultiply` that takes a `String` named `str` and an `int` named `num` as parameters and returns the string `str` with each letter copied `num` times. If `num` is 0 or less, return the empty `String`. Your solution ***must be recursive***. For example:

```
charMultiply("star", 2)    =>    "ssttaarr"  
charMultiply("c", 3)      =>    "ccc"  
charMultiply("science", 0) =>    ""
```

9. **Grids (10 points).** What does the following code draw on the *square* grid below?

```
int row = grid.getNumRows();
int col = grid.getNumCols();
int mid = col / 2;

for (int j = 0; j < row; j++)
{
    grid.setColor(j, mid, Color.GRAY);
}

for (int z = 0; z < col; z++)
{
    grid.setColor(row - 1, z, Color.GRAY);
}

for (int i = 0; i < row - 1; i++)
{
    grid.setColor(i, mid - i, Color.GRAY);
    grid.setColor(i, mid + i, Color.GRAY);
}
```


10. Turtles (10 points). Consider the following Turtle function.

```
public static void mystery(Turtle t, int n) {  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < 4; j++) {  
            t.forward(100);  
            t.turn(90);  
        }  
        t.penUp();  
        t.backward(10);  
        t.turn(90);  
        t.forward(10);  
        t.turn(-90);  
        t.penDown();  
    }  
}
```

- a. What does this function draw when the actual parameter n is 1?



b. What does this function draw when the actual parameter is 4?

